

EXTI & NVIC TRÊN STM32

1. INTERRUPT - NGẮT LÀ GÌ?

Khi có sự kiện ngắt được gửi từ một bộ ngoại vi bất kỳ nào đó lên CPU (cụ thể là bộ **NVIC** ở bên trong lõi ARM), ARM sẽ tạm ngừng thực thi mã trong hàm `main()` để nhảy tới một hàm đặc biệt gọi là **Hàm phục vụ ngắt (Interrupt Handler / ISR)**. Sau khi thực thi xong hàm `handler`, CPU sẽ quay lại tiếp tục công việc đang dang dở ở `main()`.

NGUYÊN LÝ CỐT LÕI ĐỂ TẠO RA NGẮT

Cần nhấn mạnh rằng, để một ngắt có thể thực sự diễn ra, **bắt buộc phải hội tụ đủ 2 yếu tố:**

1. **Interrupt Signal (Tín hiệu ngắt):** Phải có một bộ ngoại vi (Peripheral) nào đó tạo ra tín hiệu ngắt và gửi lên hệ thống. (Ví dụ: Bộ EXTI bắt được sự thay đổi điện áp từ nút nhấn và phát ra tín hiệu).
2. **NVIC Acceptance (Sự chấp nhận của NVIC):** Bộ điều khiển ngắt NVIC bên trong lõi ARM phải được cấu hình để **mở (enable)** và **chấp nhận** tín hiệu ngắt từ đúng bộ ngoại vi đó. Nếu ngoại vi gửi tín hiệu lên nhưng NVIC đang "đóng cửa" (mask) với đường ngắt đó, thì ngắt cũng sẽ không xảy ra.

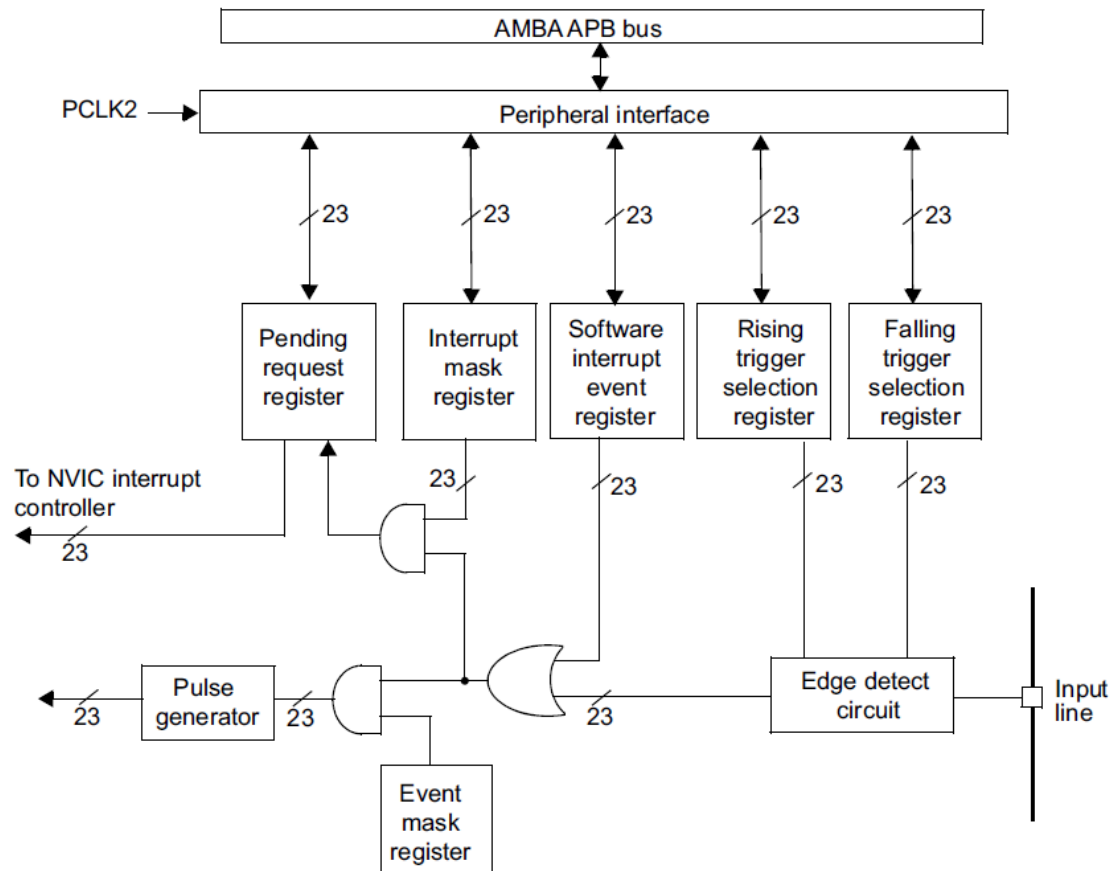
Sự khác biệt so với Polling (Hỏi vòng):

Thay vì vi điều khiển phải liên tục đọc trạng thái nút nhấn trong vòng lặp `while(1)` gây lãng phí tài nguyên, ta cấu hình để nút nhấn tự động báo cho vi điều khiển biết khi nào nó được nhấn.

2. YẾU TỐ 1: EXTI - EXTERNAL INTERRUPT (TẠO TÍN HIỆU NGẮT)

Sử dụng EXTI để đọc nút nhấn: Khi trạng thái chân GPIO nối với nút nhấn thay đổi (từ 0 lên 1 hoặc từ 1 xuống 0), EXTI sẽ bắt tín hiệu này và gửi **Interrupt signal** lên NVIC.

Để cấu hình EXTI (Yếu tố 1), chúng ta cần:



1. **Edge Detect (Cạnh kích hoạt):** Chọn ngắt xảy ra khi tín hiệu đi lên (Rising), đi xuống (Falling), hay cả hai.
2. **Interrupt Mask:** Bỏ mặt nạ ngắt (Enable) để đường EXTI đó được phép phát tín hiệu.

```
#define EXTI_BASE_ADDR 0x40013C00
#define SYSCFG_BASE_ADDR 0x40013800

void exti0_init()
{
    // 1. Map chân PA0 vào EXTI0 thông qua thanh ghi SYSCFG_EXTICR1
    uint32_t* SYSCFG_EXTICR1 = (uint32_t*)(SYSCFG_BASE_ADDR + 0x08);
    *SYSCFG_EXTICR1 &= ~(0xF << 0);

    // 2. Chọn Rising và Falling edge
    uint32_t* EXTI_RTSR = (uint32_t*)(EXTI_BASE_ADDR + 0x0
```

```

8);
*EXTI_RTISR |= (1 << 0);
uint32_t* EXTI_FTSR = (uint32_t*)(EXTI_BASE_ADDR + 0x0
C);
*EXTI_FTSR |= (1 << 0);

// 3. Enable EXTI0 - Cho phép EXTI0 tạo Interrupt Signa
l
uint32_t* EXTI_IMR = (uint32_t*)(EXTI_BASE_ADDR + 0x0
0);
*EXTI_IMR |= (1 << 0);

// ... (Code cấu hình NVIC ở phần sau)
}

```

3. YẾU TỐ 2: NVIC (CHẤP NHẬN NGẮT)

Sau khi ngoại vi đã tạo ra tín hiệu ngắt, tín hiệu này được đưa đến NVIC. Nhiệm vụ của NVIC là kiểm duyệt và quyết định có cho phép ngắt đó đứt ngang CPU hay không. Để cấu hình NVIC, chúng ta luôn đi qua 2 bước tiêu chuẩn sau:

Bước 1: Xác định Position của ngắt trong Vector Table

Mỗi một ngoại vi trên STM32 khi gửi tín hiệu ngắt đều được cấp một "số báo danh" cố định, gọi là **Position**.

Table 37. Vector table for STM32F411xC/E (continued)

Position	Priority	Type of priority	Acronym	Description	Address
	-2	fixed	NMI	Non maskable interrupt, Clock Security System	0x0000 0008
	-1	fixed	HardFault	All class of fault	0x0000 000C
	0	settable	MemManage	Memory management	0x0000 0010
	1	settable	BusFault	Pre-fetch fault, memory access fault	0x0000 0014
	2	settable	UsageFault	Undefined instruction or illegal state	0x0000 0018
	-	-	-	Reserved	0x0000 001C - 0x0000 002B
	3	settable	SVCall	System Service call via SWI instruction	0x0000 002C
	4	settable	Debug Monitor	Debug Monitor	0x0000 0030
		-	-	Reserved	0x0000 0034
	5	settable	PendSV	Pendable request for system service	0x0000 0038
	6	settable	Systick	System tick timer	0x0000 003C
0	7	settable	WWDG	Window Watchdog interrupt	0x0000 0040
1	8	settable	EXTI16 / PVD	EXTI Line 16 interrupt / PVD through EXTI line detection interrupt	0x0000 0044
2	9	settable	EXTI21 / TAMP_STAMP	EXTI Line 21 interrupt / Tamper and TimeStamp interrupts through the EXTI line	0x0000 0048
3	10	settable	EXTI22 / RTC_WKUP	EXTI Line 22 interrupt / RTC Wakeup interrupt through the EXTI line	0x0000 004C
4	11	settable	FLASH	Flash global interrupt	0x0000 0050
5	12	settable	RCC	RCC global interrupt	0x0000 0054
6	13	settable	EXTI0	EXTI Line0 interrupt	0x0000 0058
7	14	settable	EXTI1	EXTI Line1 interrupt	0x0000 005C

- **Cách tìm:** Bạn mở tài liệu **Reference Manual** của dòng chip đang dùng, tìm đến mục *Interrupt and exception vector table* (Bảng vector ngắt).
- **Mục đích:** Tìm xem ngoại vi bạn đang dùng nằm ở dòng nào và giống sang cột **Position** để lấy giá trị.
- *Ví dụ:* Ngắt của EXTI0 nằm ở Position **6**.
- *Ví dụ:* Ngắt của cụm EXTI15_10 (từ chân 10 đến chân 15) nằm ở Position **40**.

Bước 2: Cấu hình thanh ghi NVIC_ISERx để kích hoạt

NVIC quản lý việc cho phép/từ chối ngắt thông qua nhóm thanh ghi **ISER (Interrupt Set-Enable Register)**.

Bởi vì nhân ARM Cortex-M là vi xử lý 32-bit, nên mỗi thanh ghi ISER chỉ có 32 bit, tương ứng quản lý được 32 Position ngắt. Do STM32 có rất nhiều ngắt (lên tới gần 100), hệ thống chia ISER thành nhiều thanh ghi: **ISER0**, **ISER1**, **ISER2**, v.v.

Table 4-2 NVIC register summary

Address	Name	Type	Required privilege	Reset value	Description
0xE000E100-0xE000E11C	NVIC_ISER0-NVIC_ISER7	RW	Privileged	0x00000000	<i>Interrupt Set-enable Registers on page 4-4</i>
0xE000E180-0xE000E19C	NVIC_ICER0-NVIC_ICER7	RW	Privileged	0x00000000	<i>Interrupt Clear-enable Registers on page 4-5</i>
0xE000E200-0xE000E21C	NVIC_ISPR0-NVIC_ISPR7	RW	Privileged	0x00000000	<i>Interrupt Set-pending Registers on page 4-5</i>
0xE000E280-0xE000E29C	NVIC_ICPR0-NVIC_ICPR7	RW	Privileged	0x00000000	<i>Interrupt Clear-pending Registers on page 4-6</i>
0xE000E300-0xE000E31C	NVIC_IABR0-NVIC_IABR7	RW	Privileged	0x00000000	<i>Interrupt Active Bit Registers on page 4-7</i>
0xE000E400-0xE000E4EF	NVIC_IPR0-NVIC_IPR59	RW	Privileged	0x00000000	<i>Interrupt Priority Registers on page 4-7</i>
0xE000EF00	STIR	WO	Configurable ^a	0x00000000	<i>Software Trigger Interrupt Register on page 4-8</i>

CÔNG THỨC CHỌN THANH GHI VÀ BIT (RẤT QUAN TRỌNG)

Khi đã biết Position ở Bước 1, bạn dùng công thức sau để xác định cần ghi vào thanh ghi nào và ở bit số mấy:

1. **Tìm thanh ghi (Phép chia lấy nguyên):** $x = \text{Position} / 32$

- Nếu $x = 0 \rightarrow$ Dùng thanh ghi **ISER0** (Quản lý Position 0 - 31)
- Nếu $x = 1 \rightarrow$ Dùng thanh ghi **ISER1** (Quản lý Position 32 - 63)
- Nếu $x = 2 \rightarrow$ Dùng thanh ghi **ISER2** (Quản lý Position 64 - 95)

1. **Tìm vị trí Bit cần Set (Phép chia lấy dư):** $\text{Bit} = \text{Position} \% 32$

Ví dụ áp dụng thực tế:

Trường hợp 1: Kích hoạt ngắt cho EXTI0 (Position = 6)

- **Thanh ghi:** $6/32 = 0 \rightarrow$ Dùng **ISER0**
- **Vị trí bit:** $6\%32 = 6 \rightarrow$ Cần dịch trái 6 bit ($1 \ll 6$)
- **Code C:**

```
uint32_t* ISER0 = (uint32_t*) 0xE000E100; // Địa chỉ base của ISER0
*ISER0 |= (1 << 6);
```

Trường hợp 2: Kích hoạt ngắt cho EXTI15_10 (Position = 40)

- Thanh ghi: $40/32 = 1 \rightarrow$ Dùng `ISER1`
- Vị trí bit: $40\%32 = 8 \rightarrow$ Cần dịch trái 8 bit (`1 << 8`)
- Code C:

```
uint32_t* ISER1 = (uint32_t*) 0xE000E104; // Địa chỉ base của ISER1 (cộng thêm 4 byte)
*ISER1 |= (1 << 8);
```

4. HÀM PHỤC VỤ NGẮT (ISR)

Khi cả 2 yếu tố trên đã thỏa mãn, CPU sẽ nhảy vào hàm ngắt.

Lưu ý: Bắt buộc phải xóa **Cờ ngắt (Pending Bit)** trong thanh ghi `EXTI_PR` để báo cho NVIC biết là ngắt đã được xử lý xong, tránh bị kẹt trong ngắt mãi mãi.

```
void button_handler()
{
    if(button_get_state() == 1) led_control(RED_LED, 1);
    else led_control(RED_LED, 0);
}

// Hàm phục vụ ngắt của EXTI0
void EXTI0_IRQHandler(void)
{
    uint32_t* EXTI_PR = (uint32_t*)(EXTI_BASE_ADDR + 0x14);

    // Kiểm tra xem ngắt có thực sự do EXTI0 gây ra không
    if(*EXTI_PR & (1 << 0))
    {
        button_handler();
    }
}
```

```
// XÓA CỜ NGẮT bằng cách ghi 1 vào bit tương ứng
*EXTI_PR |= (1 << 0);
}
}
```